HW3 Report — Simple MPI Circle Renderer

Team 14

Members: R14944049 吳冠磊、R14942037 李洋、R11922119 陳聖諺

Datasets

Dataset	Circles	Image size	File size
`imbalance_c100000`	100,000	320×240	~1.4 MB
`medium_c200000`	200,000	640×480	~2.9 MB
`large_c1000000`	1,000,000	640×480	~14.3 MB
`large_c2000000`	2,000,000	640×480	~28.6 MB
`large_c4000000`	4,000,000	640×480	~57.2 MB

The `imbalance` dataset uses a 320×240 canvas — one quarter of the pixel area — with a skewed radius distribution designed to stress load-balancing algorithms. The remaining datasets all render to 640×480.

Runtime is dominated by pixel fill: a circle of radius r touches up to $\pi \cdot r^2$ pixels, so a few large circles cost as much as thousands of small ones.

Serial Baseline

Measured on a single core (median of three runs):

Dataset	Circles	Render time (s)
`imbalance_c100000`	100K	0.223
`medium_c200000`	200K	0.385
`large_c1000000`	1M	1.065
`large_c2000000`	2M	~2.11*
`large_c4000000`	4M	4.199

* The 2M run showed one outlier (2.76 s); the 2.11 s figure is the minimum of three runs.

Runtime scales roughly linearly with circle count when the radius distribution is uniform (200K → 1M = 5× circles, ~2.8× time), reflecting the mix of small and large circles. The imbalance dataset is faster per circle because its smaller canvas (320×240) means fewer pixels are tested.

Strong Scaling

Dataset: `large_c1000000` (1M circles, 640×480)

Serial baseline (renderer.c): **1.065 s**

Processes	Wall time (s)	Render time min/max/avg (s)	Speedup	Efficiency
1	2.056	1.982 / 1.982 / 1.982	1.00×	100%
2	0.994	0.882 / 0.911 / 0.897	2.07×	103%
4	0.604	0.442 / 0.511 / 0.474	3.40×	85%
8	0.401	0.222 / 0.305 / 0.253	5.13×	64%
16	0.285	0.112 / 0.167 / 0.121	7.21×	45%

> Speedup is relative to the MPI 1-process wall time (2.056 s). Efficiency = speedup / nprocs.

Dataset: `large_c4000000` (4M circles, 640×480)

Serial baseline (renderer.c): **4.199 s**

Processes	Wall time (s)	Render min/max/avg (s)	Speedup	Efficiency
1	5.877	5.739 / 5.739 / 5.739	1.00×	100%
2	3.270	2.144 / 3.124 / 2.634	1.80×	90%
4	1.932	1.072 / 1.750 / 1.558	3.04×	76%
8	1.054	0.526 / 0.877 / 0.745	5.57×	70%
16	0.641	0.265 / 0.454 / 0.403	9.17×	57%

Dataset: `medium_c200000` (200K circles, 640×480)

Processes	Wall time (s)	Speedup
1	0.413	1.00×
2	0.247	1.67×
4	0.157	2.63×
8	0.133	3.10×
16	0.132	3.13×

The smaller 200K dataset saturates at ~8 processes because communication overhead (scatter + gather of a 640×480×3×4 = ~3.7 MB float buffer) becomes comparable to compute time.

Key observations:

- The MPI 1-process baseline includes I/O and communication setup and is ~1.9× slower than the pure serial renderer for 1M circles, explaining super-linear speedups at p=2.
- For large datasets (4M), efficiency at p=16 is still 57%, a reasonable result given the sequential gather phase on rank 0.
- The gather/composite on rank 0 is the primary bottleneck at high process counts.

Weak Scaling

Weak scaling keeps the work per process approximately constant at ~100,000 circles per process.

Processes	Dataset	Circles/proc	Per-rank render avg (s)	Imbalance
1	imbalance_c100000	100K	0.210	0.00%
2	medium_c200000	100K	0.187	1.56%
10	large_c1000000	100K	0.212	18.26%
16	large_c2000000	125K	0.223	0.39%

The local render time remains near 0.21–0.22 s across all configurations, demonstrating **near-ideal weak scaling** in the compute phase. The per-rank render time does not grow with process count, confirming that the algorithm scales linearly with circles per process.

The elevated imbalance at 10 processes (18.26%) is partly due to the weight-based partition boundary effects — with 10 ranks, the greedy weight assignment produces slightly uneven cuts near the tail of the distribution.

Load Imbalance Analysis

Regular datasets

For the uniform-radius `large_c1000000` dataset:

Processes	min (s)	max (s)	avg (s)	Imbalance (max/avg − 1)
2	0.882	0.911	0.897	**1.6%**
4	0.442	0.511	0.474	**7.8%**
8	0.222	0.305	0.253	**20.6%**
16	0.112	0.167	0.121	**37.6%**

Imbalance grows with process count because finer partitions expose boundary effects in the weight-based greedy assignment. Additionally, random variation in circle overlap (multiple circles hitting the same pixel region) creates non-linear cost interactions that the r^2-weight heuristic cannot fully account for.

Imbalance dataset

The `imbalance_c100000` dataset uses a deliberately skewed radius distribution:

Processes	min (s)	max (s)	Imbalance
2	0.102	0.116	6.6%
4	0.058	0.070	11.7%
8	0.031	0.048	29.1%
16	0.022	0.053	**68.1%**

At p=16, the maximum rank takes 2.4× longer than the minimum, indicating significant load imbalance. Despite the r^2-weighted partition, a few very large circles can dominate the cost of a rank's assigned slice, while most ranks finish quickly. A finer-grained assignment (e.g., work-stealing or 2D spatial decomposition) would be necessary to fully eliminate this.

Optimization

We developed four progressive optimizations of the MPI renderer (`renderer_mpi_modify.c` through `renderer_mpi_modify4.c`). This section benchmarks each variant and analyzes the impact of each change.

All benchmarks were run locally (`mpirun` on one node) with `-O3 -march=native`. Timings are single-run wall times. Serial reference: `renderer.c` compiled with `-O3 -march=native`.

Serial Baseline

The serial `renderer.c` is shared across all variants. Timings below are used as the speedup reference throughout this section.

Dataset	Serial time (s)
imbalance_c100000	0.207
medium_c200000	0.364
large_c1000000	1.065
large_c2000000	1.993
large_c4000000	3.999

Algorithm Progression

Version	Per-pixel buffer	Gather method	Rasterization
`renderer_mpi`	`float[W×H×3]` (24 B/px)	Sequential `MPI_Recv` loop	Per-pixel `dx²+dy² ≤ r²`
`modify`	`uint64[W×H]` (8 B/px)	`MPI_Reduce(MPI_MAX)`	Per-pixel (incremental dy^2)
`modify2`	`uint64[W×H]` (8 B/px)	`MPI_Reduce(MPI_MAX)`	Analytical x-interval per row
`modify3`	`uint32` or `uint64` (4–8 B/px)	`MPI_Reduce(MPI_MAX)`	Analytical + 8× unrolled fill_span
`modify4`	`uint32` or `uint64` (4–8 B/px)	`MPI_Reduce(MPI_MAX)`	Analytical + branchless floor/ceil + incremental dy^2

Key insight — `modify`: Packs (rank_id << 32 | RGB) into a `uint64_t` per pixel so that `MPI_Reduce(MPI_MAX)` implicitly selects the pixel from the highest-ranked process, which is the latest-drawn circle. This replaces the O(p) sequential receive loop with a single parallel collective.

Key insight — `modify2`: For each scanline y, solves `dx_lim = sqrt(r² - dy²)` analytically and fills the entire x-span without per-pixel distance checks. This is the most impactful single algorithmic improvement.

Key insight — `modify3`: When `nprocs ≤ 255`, uses `uint32_t` instead of `uint64_t`, halving the `MPI_Reduce` bandwidth and per-pixel memory.

Strong Scaling — `large_c4000000` (4M circles, 640×480)

Serial baseline (renderer.c): **4.20 s**

Processes	renderer_mpi (s)	modify (s)	modify2 (s)	modify3 (s)	modify4 (s)
1	5.93	4.35	3.23	2.64	2.92
2	3.14	3.50	1.72	1.44	1.54
4	1.89	1.91	0.96	0.78	0.85
8	1.04	1.09	0.59	0.52	0.53
16	0.67	0.66	0.39	**0.35**	0.38

Speedup relative to serial (4.20 s) at p=16:

renderer_mpi	modify	modify2	modify3	modify4
6.3×	6.4×	10.8×	**12.0×**	11.1×

`modify3` achieves the best result on 4M circles at p=16 (**12.0× over serial**, 0.349 s), primarily driven by the analytical rasterization (`modify2` step) and 32-bit reduce bandwidth reduction.

Strong Scaling — `large_c1000000` (1M circles, 640×480)

Serial baseline (renderer.c): **1.065 s**

Processes	renderer_mpi (s)	modify (s)	modify2 (s)	modify3 (s)	modify4 (s)
1	1.83	1.84	0.83	0.68	0.74
2	1.06	0.96	0.45	0.38	0.41
4	0.57	0.52	0.27	0.22	0.30
8	0.41	0.35	0.20	0.17	0.19
16	0.30	0.22	0.14	**0.12**	0.14

Speedup at p=16:

renderer_mpi	modify	modify2	modify3	modify4
3.6×	4.8×	7.6×	**8.9×**	7.6×

`modify3` again leads at p=16 (0.120 s, 8.9× speedup). Note that `modify2` and `modify4` are essentially tied — the branchless floor/ceil in `modify4` offers no additional benefit at this scale.

Strong Scaling — `imbalance_c100000` (100K circles, 320×240)

Processes	renderer_mpi (s)	modify (s)	modify2 (s)	modify3 (s)	modify4 (s)
1	0.226	0.297	**0.072**	0.068	0.069
2	0.144	0.174	0.071	0.059	0.062
4	0.098	0.139	0.069	0.057	0.059
8	0.108	0.095	0.063	0.052	0.055
16	0.092	0.076	0.056	**0.054**	0.057

On the small imbalanced dataset the communication overhead dominates at high p, and all optimized variants plateau. `modify3` at p=8–16 (≈52–54 ms) is the fastest, matching `modify4`. `renderer_mpi` at p=16 (92 ms) is significantly slower due to the sequential gather overhead being relatively larger at small problem sizes.

Per-Rank Load Balance — `modify4`, `large_c4000000`

Processes	min (s)	max (s)	avg (s)	Imbalance (max/avg − 1)
2	1.374	1.393	1.383	**0.7%**
4	0.689	0.702	0.696	**0.7%**
8	0.341	0.379	0.357	6.2%
16	0.170	0.210	0.178	17.5%

The analytical rasterization in `modify4` produces much better load balance at low process counts (0.7% at p=2,4) compared to `renderer_mpi` (18.6% at p=2). At p=16 the imbalance grows to 17.5%, the same trend, indicating the weight-based partition remains the limiting factor at fine granularity.

Head-to-Head Summary at p=16

Dataset	renderer_mpi (s)	renderer_mpi_modify3 (s)	improvement
imbalance_c100000	0.092	0.054	**1.7×**
large_c1000000	0.303	0.120	**2.5×**
large_c4000000	0.667	0.349	**1.9×**

`renderer_mpi_modify3` is consistently the fastest variant at p=16, achieving **1.7×–2.5× faster** than `renderer_mpi` through two complementary improvements: analytical rasterization (eliminating per-pixel distance tests) and 32-bit `MPI_Reduce` (halving collective bandwidth).

Reflections & What We Learned (Anything to share)

During the development of this MPI pixel renderer, we encountered severals interesting challenges and insights:

1. **Rethinking Communication (The `MPI_MAX` Trick):** Our most satisfying breakthrough was the `modify` version, where we packed the rank ID and RGB values into a single `uint64_t` (and later `uint32_t`). This allowed us to replace a sequential O(p) `MPI_Recv` loop with a highly optimized O(log p) tree-based `MPI_Reduce(MPI_MAX)`. It elegantly solved the depth-sorting requirement (latest rank "wins" the pixel) directly within the MPI collective operation.

2. **Algorithmic Math Beats Brute Force:** While parallelization gave us compute scaling, the analytical x-interval calculation (`modify2`) provided the largest single leap in performance (often doubling local render speed). Solving $dx = \sqrt{r^2 - dy^2}$ proved that optimizing the serial core algorithm remains just as critical as MPI tuning.

3. **The Limits of Static Load Balancing:** Our tests on the `imbalance_c100000` dataset painfully exposed the limits of static, weight-based partitioning. Even when weighting by area (r^2), a single rank getting stuck with a few massive circles caused up to 68% imbalance at 16 processes. **Future work** would definitely involve a dynamic load-balancing approach, such as a master-worker task queue, or implementing a 2D spatial tile decomposition where dense regions are finely subdivided.

4. **Communication Bound at Scale:** For smaller resolutions and datasets, passing the entire framebuffer over MPI quickly becomes the bottleneck. If we were to optimize further for a single multicore node, a hybrid **MPI + OpenMP** approach would likely be superior, avoiding the massive memory copying overhead that purely distributed MPI suffers from.